

ADMISSION MANAGEMENT SYSTEM

Installation & Configuration Guide

A MERN-Stack Deployment Reference — Phase 1: Installation, Phase 2: Configuration

Document Version	1.0
Applies To	Backend v1.0.0 (Node.js / Express) · Frontend (React 19 / Vite)
Prepared For	Local development, staging, and production deployment
Author	Fateh Noor — Sukkur IBA University
Date	July 2026

Contents

Overview & Prerequisites

PHASE 1 — INSTALLATION

- 1.1 Obtaining the Source Code
- 1.2 Installing Backend Dependencies
- 1.3 Installing Frontend Dependencies
- 1.4 Verifying the Installation

PHASE 2 — CONFIGURATION

- 2.1 Backend Environment Variables
- 2.2 Database Configuration (MongoDB)
- 2.3 JWT & Authentication Configuration
- 2.4 Email (SMTP) Configuration
- 2.5 Frontend API Configuration
- 2.6 CORS Configuration
- 2.7 Seeding Demo Data

Running the Application (Development)

Production Deployment Notes

Troubleshooting

Overview & Prerequisites

This guide walks through deploying the Admission Management System from a fresh clone to a fully configured, running instance. The deployment process is split into two phases, matching how the project must actually be set up: **Phase 1 (Installation)** gets the code and dependencies onto the machine; **Phase 2 (Configuration)** connects the application to a real database, mail server, and the environment it will run in. Installation must be completed before configuration, since the configuration files being edited in Phase 2 (.env, service URLs) only exist once the corresponding `npm install` has been run.

System Requirements

- **Node.js** 18.x or later — both backend and frontend run on Node/npm
- **npm** 9.x or later (bundled with Node.js)
- **MongoDB** — a local instance, or a free MongoDB Atlas cluster
- **Git** — to clone the repository
- An SMTP-capable email account (e.g., Gmail with an App Password) for password-reset and notification emails
- 4 GB RAM minimum, and a stable internet connection for npm / Atlas access

Project Layout

The repository contains two independent Node projects that are installed and configured separately:

```
admission-management-system/
|-- backend/           Express API, Mongoose models, file uploads
|   |-- config/        Database connection
|   |-- controllers/   Route handlers (auth, applications, courses, ...)
|   |-- models/        User, Course, Application, Notification, Settings
|   |-- routes/        REST endpoints
|   |-- utils/         seed.js, generatePDF.js, sendEmail.js
|   |-- server.js
|-- frontend/          React 19 + Vite single-page application
    |-- src/
```

PHASE 1

INSTALLATION

Get the source code and all dependencies onto your machine.

1.1 Obtaining the Source Code

1 Clone the repository

```
git clone https://github.com/yourusername/admission-management-system.git
cd admission-management-system
```

1.2 Installing Backend Dependencies

1 Move into the backend directory and install packages

```
cd backend
npm install
```

This installs Express, Mongoose, JWT, bcryptjs, Multer, Nodemailer, ExcelJS, and PDFKit as listed in `backend/package.json`, along with the nodemon dev dependency used for hot-reloading.

2 Confirm the backend installed cleanly

```
npm list --depth=0
```

This should print the top-level dependency tree with no UNMET DEPENDENCY warnings.

1.3 Installing Frontend Dependencies

1 Move into the frontend directory and install packages

```
cd ../frontend
npm install
```

This installs React 19, React Router 7, Axios, Tailwind CSS 4, Bootstrap, SweetAlert2, and the Vite build tooling.

1.4 Verifying the Installation

1

Start each project in isolation to confirm it boots

From backend/:

```
npm run dev
```

At this point the server will attempt to start but will fail to connect to a database and will not yet send email — this is expected. Installation is complete once the process starts without missing-module errors; the connection and email errors are resolved in Phase 2.

Installation vs. Configuration

A server process that starts but immediately logs a MongoDB connection error is a sign that **installation succeeded** — all packages resolved — but **configuration is still pending**. Do not troubleshoot missing environment variables as an installation problem; proceed to Phase 2.

PHASE 2

CONFIGURATION

Connect the installed application to a database, mail server, and its runtime environment.

2.1 Backend Environment Variables

The backend reads all configuration from a `.env` file in `backend/`, loaded via `dotenv`. This file is intentionally excluded from version control (see `.gitignore`), so it must be created manually on every environment.

1 Create `backend/.env` with the following keys

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/admission_system
JWT_SECRET=change_this_to_a_long_random_secret_key
JWT_EXPIRE=7d
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password
FRONTEND_URL=http://localhost:5173
```

Variable Reference

Variable	Purpose	Notes
PORT	Port the Express server listens on.	Default 5000; Railway overrides this automatically in production.
MONGO_URI	MongoDB connection string.	See §2.2 for local vs. Atlas values.
JWT_SECRET	Signing key for auth tokens.	Must be a long, random string — never reuse the placeholder value.
JWT_EXPIRE	Token lifetime.	Accepts values like 7d, 12h, 30m (jsonwebtoken format).
EMAIL_HOST / EMAIL_PORT	SMTP server address and port.	587 for TLS (STARTTLS); 465 for implicit SSL.
EMAIL_USER / EMAIL_PASS	SMTP account credentials.	See §2.4 for Gmail App Password setup.
FRONTEND_URL	Base URL used to build password-reset links.	Must match wherever the frontend is actually served.

Behavior when EMAIL_USER / EMAIL_PASS are unset

The backend's `sendEmail` utility checks for these two variables before sending. If either is missing, it silently skips sending and logs the message to the console instead — useful for local development, but it means password-reset and notification emails will not actually be delivered until real SMTP credentials are set.

2.2 Database Configuration (MongoDB)

The backend connects via Mongoose using whatever value `MONGO_URI` holds — there is no other database configuration to edit in code.

1 Option A — Local MongoDB (development)

Install and start MongoDB locally, then use:

```
MONGO_URI=mongodb://127.0.0.1:27017/admission_system
```

2 Option B — MongoDB Atlas (recommended for staging/production)

Create a free cluster at MongoDB Atlas, add a database user, allow network access from your deployment environment's IP (or `0.0.0.0/0` for Railway), and copy the connection string into `MONGO_URI`:

```
MONGO_URI=mongodb+srv://<user>:<pass>@<cluster>.mongodb.net/admission_system
```

3 Confirm the connection

Start the backend (`npm run dev`) and check the console for `MongoDB Connected: <host>`. A connection failure exits the process — check the URI, credentials, and Atlas network-access rules.

2.3 JWT & Authentication Configuration

1 Generate a strong JWT secret

Never deploy with the placeholder secret. Generate a random 64-character value, for example:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Paste the output into `JWT_SECRET` in `backend/.env`.

2 Set an appropriate token lifetime

`JWT_EXPIRE` controls how long a login session remains valid before the user must sign in again. The default of 7d is reasonable for a student-facing portal; shorten it for stricter administrative sessions if needed.

2.4 Email (SMTP) Configuration

Email is used for two flows: password-reset links (\$Auth) and manual/status-change notifications sent to students. Both go through the same `sendEmail` utility (Nodemailer).

1 Using Gmail (as pre-configured in .env)

Gmail requires an App Password rather than your normal account password:

- Enable 2-Step Verification on the Google account.
- Go to Google Account → Security → App passwords, and generate one for "Mail".
- Set `EMAIL_USER` to the full Gmail address and `EMAIL_PASS` to the generated app password (not the account password).

2 Using another SMTP provider

Any SMTP provider works — update `EMAIL_HOST` and `EMAIL_PORT` to match (e.g., `smtp.mailgun.org`, `smtp.sendgrid.net`), and set secure mode correctly: port 465 uses implicit TLS, port 587 uses STARTTLS. The backend already selects this automatically based on `EMAIL_PORT == 465`.

2.5 Frontend API Configuration

The frontend's Axios instance (`frontend/src/services/api.js`) currently points to a **hard-coded** base URL rather than reading from an environment variable:

```
const API = axios.create({
  baseURL: "http://localhost:5000/api",
  // production baseURL:
  // "https://admission-management-system-production-06f1.up.railway.app/api",
  ...
});
```

1 For local development

No change needed — the default already points at the local backend on port 5000.

2 For staging or production

Update `baseURL` to the deployed backend's public URL before building the frontend. Because Vite embeds this value at build time, the frontend must be rebuilt (`npm run build`) after any change.

Recommended improvement

Hard-coding the base URL means every environment requires a source change and rebuild. Consider migrating to a Vite environment variable (e.g., `VITE_API_URL` read via `import.meta.env.VITE_API_URL`) so staging and production builds can be configured without editing `api.js` directly.

2.6 CORS Configuration

The backend's CORS policy in `server.js` is an explicit allow-list of origins, not a wildcard:

```
app.use(
  cors({
    origin: [
      "http://localhost:5173",
      "https://admission-management-system-alpha.vercel.app"
    ],
    credentials: true,
  })
);
```

1 Add every frontend origin you deploy to this list

If you deploy the frontend under a different Vercel project name or a custom domain, add that exact origin (scheme + host, no trailing slash) to the array in `backend/server.js` and redeploy the backend — requests from an origin not in this list will be blocked by the browser.

2.7 Seeding Demo Data

The backend includes a seed script (`backend/utils/seed.js`) that clears existing data and creates a Super Admin, an Admission Officer, a demo Student, five sample courses, and ten sample applications in various statuses — useful for a fresh install or a demo environment.

1

Run the seed script

From `backend/`, with `MONGO_URI` already configured:

```
npm run seed
```

Destructive operation

The seed script deletes all existing Users, Courses, Applications, and Settings before inserting fresh sample data. Do not run it against a database containing real applicant data.

Seeded Accounts

Role	Email	Password
Super Admin	superadmin@college.com	Admin@123
Admission Officer	officer@college.com	Officer@123
Student	student@gmail.com	Student@123

Change or remove these credentials before any public-facing deployment.

Running the Application (Development)

With both installation and configuration complete, run the backend and frontend in two terminals:

1 Start the backend API

```
cd backend
npm run dev
```

Runs on `http://localhost:5000` via nodemon, restarting on file changes.

2 Start the frontend

```
cd frontend
npm run dev
```

Runs on `http://localhost:5173` via the Vite dev server.

3 Log in

Open `http://localhost:5173` and sign in with one of the seeded accounts from §2.7, or register a new student account.

Production Deployment Notes

The project is structured for the frontend to deploy on Vercel and the backend on Railway, with MongoDB Atlas as the managed database.

Component	Build Command	Start / Output
Backend (Railway)	<code>npm install</code>	<code>npm start</code> → runs <code>server.js</code> on <code>process.env.PORT</code>
Frontend (Vercel)	<code>npm run build</code>	Static output in <code>frontend/dist</code> , served by Vercel

- Set every variable from §2.1 in the hosting platform's environment-variable settings — the `.env` file itself is not deployed.
- Update `FRONTEND_URL` to the production frontend URL so password-reset emails link correctly.
- Update the CORS origin list (§2.6) and the frontend's `baseUrl` (§2.5) to match the real deployed URLs before going live.
- Uploaded files are currently written to local disk (`backend/uploads/`). On platforms with an ephemeral filesystem, plan for persistent or cloud storage (see Future Enhancements in the project SRS).

Troubleshooting

Symptom	Likely Cause	Fix
Server exits with a MongoDB connection error	MONGO_URI missing, malformed, or unreachable	Re-check §2.2; confirm Atlas network access / local MongoDB is running
Login succeeds but subsequent requests return 401	JWT_SECRET differs between requests (e.g., changed after tokens were issued)	Re-authenticate; keep JWT_SECRET stable across restarts
Password-reset emails never arrive	EMAIL_USER / EMAIL_PASS unset, or Gmail account password used instead of an App Password	Re-check §2.4; confirm console does not show "[sendEmail] Skipped"
Browser blocks API requests (CORS error)	Frontend origin not in the backend's CORS allow-list	Add the exact origin in server.js per §2.6 and redeploy the backend
Frontend calls the wrong backend after deployment	baseUrl in api.js still points at localhost	Update and rebuild per §2.5
File upload fails with a type/size error	File is not .jpg/.jpeg/.png/.pdf, or exceeds 5 MB	Expected behavior — enforced in backend/middleware/upload.js

End of guide. For the full functional and non-functional requirements of the system, refer to the accompanying Software Requirements Specification.